

Atelier — Trois produits

À ouvrir après 1-COURS.pdf du dossier projets/04-trois-produits/.

1–3 périodes. Trois produits du camion du quai.

Le dossier depart/ de cet exercice contient les fichiers de départ. Vous n'avez pas à ouvrir un terminal.

Où l'on va

Les fiches planète et les meules tenaient déjà avec une classe, des objets et une règle. Vous allez refaire le même geste dans un autre domaine : d'abord une féra seule, puis les trois produits du camion. Il n'y a ni panier, ni commande, ni paiement.

À la fin, votre écran ressemble à ceci. « Frites maison » est épuisé parce que l'objet a un stock de 0 et que la page lui demande `estDispo()`.

CAMION DU QUAI

La carteLa méthode

SAS · FOOD-TRUCK · TROIS PRODUITS

Trois produits, une règle dans la classe.

Un produit est disponible si `stock > 0`. Pas de panier. Pas de paiement. La page demande `estDispo()` à chaque objet.

Ce soir sur le camion

Féra, frites, soupe. Le stock décide.

Féra du Léman

PRIX
18 francs

STOCK
8

DISPONIBLE

Frites maison

PRIX
7 francs

STOCK
0

ÉPUISÉ

Soupe du jour

PRIX
9 francs

STOCK
3

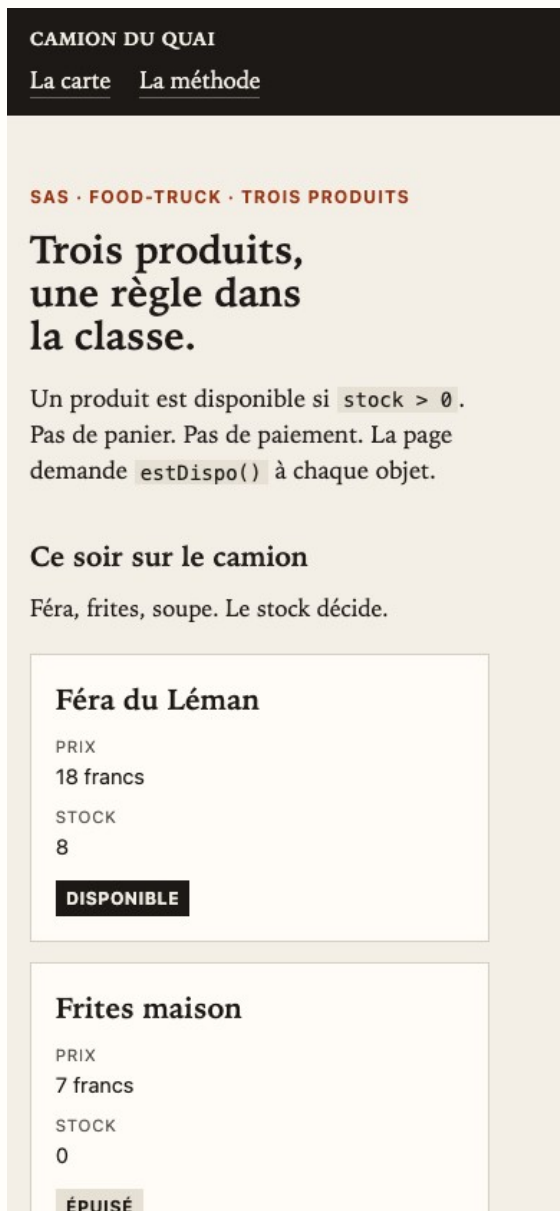
DISPONIBLE

Ce que l'objet sait

`Frites.estDispo()` répond non : `stock == 0`. Décision dans la classe.

Corrigé · classe Produit · pas de panier.

Sur un téléphone de 390 pixels de large, les cartes s'empilent et aucun mot n'est coupé.



Vous créez exactement ceci

```
sas/
├─ trois-produits/
│  └─ index.html
│     └─ css/
│        └─ style.css
│     └─ js/
│        └─ produit.js
```

Le départ contient déjà le HTML et le CSS. Votre travail se fait dans `js/produit.js` : vous complétez la règle, vous testez un objet, puis vous construisez la carte avec trois objets.

Étape 1 — Ouvrir le départ

Vous avez déjà affiché des fiches et des meules à partir d'objets. Ici, la scène change, mais la page attend le même geste : elle est prête et la zone des produits est vide tant qu'aucun objet n'existe.

Placez le départ dans `sas/trois-produits/`, puis ouvrez `sas/trois-produits/index.html` par un double-clic. Vous devez voir le camion du quai, le titre « Trois produits, une règle dans la classe » et une carte encore vide.

- ☐ L'arbre correspond exactement au dessin ci-dessus.
- ☐ Le décor du camion est déjà mis en forme.
- ☐ Aucune carte de produit n'est encore affichée.
- ☐ La console du navigateur ne montre pas d'erreur rouge.

Vous n'avez à recopier ni le HTML ni le CSS depuis ce PDF.

Étape 2 — Une féra répond

Le stock et la disponibilité appartiennent au même produit. Si le HTML écrit « Disponible » en dur, il peut contredire le stock. La méthode `estDispo()` garde donc la règle sur l'objet qui connaît `this.stock`.

Dans `js/produit.js`, remplacez le commentaire `/* à vous : une ligne */` de `estDispo()`. La règle nommée **produit disponible** est :

```
estDispo() {  
  /* à vous : une ligne */  
}
```

```
retourner vrai quand le stock est supérieur à 0
```

Sous la classe, fabriquez seulement la féra :

```
const fera = new Produit("Féra du Léman", 18, 8);
```

Rechargez, ouvrez la console et tapez :

```
fera.estDispo();
```

Vous devez obtenir `true`. `Produit` est le moule ; `fera` est un objet réel, comme un billet réel sorti d'un moule à billets.

Étape 3 — Trois objets, même question

La féra répond correctement. Remplacez `const fera = ...` par les trois objets du camion :

```
const carte = [  
  new Produit("Féra du Léman", 18, 8),  
  new Produit("Frites maison", 7, 0),
```

```
new Produit("Soupe du jour", 9, 3),
];
```

Ajoutez ensuite l’affichage. La boucle ne compare jamais directement le stock : elle demande `estDispo()` à chaque produit.

```
const liste = document.getElementById("liste");

carte.forEach((produit) => {
  const dispo = produit.estDispo();
  const article = document.createElement("article");
  article.className = "fiche";
  article.innerHTML = `
<h3>${produit.nom}</h3>
<dl>
<dt>Prix</dt><dd>${produit.prix} francs</dd>
<dt>Stock</dt><dd>${produit.stock}</dd>
</dl>
<p class="badge ${dispo ? "badge--oui" : "badge--non"}">
  ${dispo ? "Disponible" : "Épuisé"}
</p>`;
  liste.appendChild(article);
});
```

Ajoutez enfin ce contrôle sous la boucle :

```
const frites = carte[1];
document.getElementById("oral").textContent = frites.estDispo()
? "Les frites devraient être épuisées – vérifiez le stock 0."
: "Frites.estDispo() répond non : stock === 0.";
```

Rechargez. La féra et la soupe sont disponibles. Les frites sont épuisées **parce que** leur objet possède `stock === 0` ; aucun texte « Épuisé » n’est fixé dans `index.html`.

Réussi si

- `[]` Vous savez montrer la féra seule avant la liste.
- `[]` `fera.estDispo()` répond `true`.
- `[]` Trois objets `Produit` existent à la fin.
- `[]` « Frites maison » possède un stock de `0` et s’affiche comme épuisé.
- `[]` La boucle demande `estDispo()` aux objets sans recalculer `stock > 0`.
- `[]` La page se lit à 1440 et à 390 pixels de large.
- `[]` Aucun panier, aucune commande et aucun paiement n’ont été ajoutés.

Vous avez écrit `estDispo()` vous-même. Montrez seulement la boucle à une IA et demandez : « Expliquez en quatre phrases pourquoi Frites maison s'affiche comme épuisé sans que le HTML connaisse la règle. » Refusez une réponse qui propose de coder le mot « Épuisé » directement dans `index.html`.

POUR LES RAPIDES

Ajoutez un quatrième `Produit` avec un stock de 0, puis prédissez son badge avant de recharger. Changez uniquement son stock à 1 : la page doit changer de réponse sans modification du HTML ni de `estDispo()`.

Livrable

Rendez le dossier `sas/trois-produits/` avec ses trois fichiers. La page doit s'ouvrir par double-clic. À l'oral, vous montrez la féra seule, puis vous expliquez pourquoi l'affichage demande `estDispo()` à chaque objet.